

Horizon 87

Developer Architecture Guide

Module Map, Runtime Flow, Worker Pipeline & Implementation Notes

Version 1.0 - June 2026 - Desktop Web Application

This document describes the engineering architecture of Horizon 87 from the public source snapshot, focusing on module roles, runtime flow, storage, validation, Mission Lab, and rendering boundaries.

SECTION 00

Navigation Index

Sec	Section	Contents
01	Architecture Overview	Browser-native runtime, source snapshot, and separation of responsibilities.
02	Runtime Layers	UI, ephemeris, ZENITH, storage, validation, Mission Lab, and renderer layers.
03	Module Map	Purpose, runtime location, inputs, outputs, and major responsibilities of each file.
04	Startup & Engine Routing	Script load order, capability checks, fallbacks, and active ephemeris selection.
05	Precision Asset Pipeline	OPFS storage, DE441 package files, DE442s kernel, metadata, and downloader strategy.
06	ZENITH Runtime & Worker Flow	CSPICE/WASM loading, worker-first execution, synchronous fallback, request IDs, and vector calls.
07	Validation Lab Flow	Horizons reference fetch, convention matching, residual record construction, and dossier export.
08	Mission Lab Flow	Lambert grid generation, state sampling, filtering, metrics, and porkchop rendering.
09	2D UI & Observer Systems	Date handling, time conversion, engine controls, scanners, and sky-event/UI orchestration.
10	3D Renderer Flow	Texture loading, body construction, focus controls, orbit rendering, exposure, HUD, and frame timing.
11	Public Repository Boundaries	Included and excluded materials, private runtime boundaries, and reproducibility posture.
12	Engineering Constraints & Future Notes	Browser requirements, storage limits, live-reference dependency, and future extension points.

SECTION 01

Architecture Overview

Horizon 87 is organized as a browser-native technical application rather than a single-script visualizer. The codebase separates interface state, ephemeris computation, precision kernels, OPFS storage, validation, mission screening, and 3D rendering into dedicated modules. The result is a layered system where fast exploratory features remain responsive while higher-trust validation and precision paths load only when assets are available.

The architecture is intentionally client-centered. Once precision assets are installed, local state-vector calculation and Mission Lab operation can run in the desktop browser; only live-reference validation still depends on the Horizons proxy/API path.

IN PLAIN ENGLISH Horizon 87 is wired like a small desktop scientific application that happens to run in a browser: a UI shell, a rendering system, local astronomical engines, a browser-private file system, background workers, and a validation lab.

Area	Primary files	Reviewed role
Application shell	index.html, main.css, sitemap.xml	Loads runtime scripts, defines public page metadata, and hosts UI panels, canvases, engine controls, ZENITH decks, validation controls, and Mission Lab controls.
2D / observer runtime	ui.js, vsop_worker.js, tz-lookup.js, countries.geojson	Controls date/time conversion, view modes, scanner UI, geocentric/heliocentric drawing, observer presets, timezone/location helpers, and analytical/survey calculations.
Precision runtime	precision_engine.js, precision_storage.js	Manages compact DE442s precision path, local OPFS storage, date coverage checks, state-vector conversion, and Geometry Finder eclipse windows when available.
ZENITH runtime	zenith_config.js, zenith_storage.js, zenith_engine.js, zenith_worker.js, zenith_downloader_worker.js	Manages DE441 package acquisition, CSPICE/WASM loading, worker-backed state-vector calls, kernel mounting, and package health/status reporting.
Lab runtime	reloaded_labs.js	Implements validation dossiers, residual analysis, ET offset sweeps, Mission Lab, porkchop rendering, and lab-side orchestration.
3D telemetry runtime	three_mode.js	Builds the WebGL scene, body geometry, textures, labels, orbits, focus controls, and display-synced 3D telemetry.

SECTION 02

Runtime Layers

The runtime is divided into layers so that display, computation, validation, and storage are not bound to the same readiness state. A user can view survey-grade positions immediately, install precision packages later, and run validation only when both local ZENITH and the live Horizons reference path are available.

Layer	Primary responsibility	Important design boundary
Interface layer	Holds DOM controls, engine picker, date controls, sidebars, mode toggles, scanner panels, and 2D canvas drawing.	Should remain responsive even when precision packages or validation requests are not ready.
Survey ephemeris layer	Provides immediate analytical positions, scanning, and fallback behavior for broad visual exploration.	Useful for exploration; not treated as a validation certificate.
Precision layer	Loads DE442s / DE441 through OPFS and CSPICE/WASM where supported.	Readiness depends on browser capabilities, installed files, kernel coverage, and runtime loading.
ZENITH validation layer	Compares local state vectors against live NASA/JPL Horizons VECTORS rows under matched conventions.	High-trust only when target, observer, frame, time scale, and correction convention match.
Mission Lab layer	Generates departure/arrival grids, solves Lambert arcs, computes energy metrics, filters unstable cells, and renders porkchop plots.	Preliminary screening only; not spacecraft navigation.
Renderer layer	Maps active ephemeris positions into 2D and 3D display contexts.	Rendering is interpretive; it is not an independent ephemeris authority.

The architecture intentionally separates readiness from trust. A module can be visually useful before it is validation-grade, and validation can be valid without implying every scanner or display path carries the same precision claim.

SECTION 03

Module Map

This section maps the major source files to their engineering roles. It is not a full code listing; it documents the responsibilities and data movement visible from the V1 source snapshot.

Module	Runtime location	Responsibility
index.html	Main document	Defines metadata, script order, canvases, 3D container, ZENITH HUD, sidebar decks, validation controls, Mission Lab controls, and application panels.
ui.js	Main thread	Primary 2D controller: date handling, time conversions, engine selection, geocentric/heliocentric rendering, scanner controls, lunar phase/eclipses, observer guide, and UI state.
vsop_worker.js	Loaded analytical engine	Provides analytical ephemeris, sandbox/observatory modes, gravitational parameters, scanning utilities, and fallback/survey calculations.
precision_storage.js	Main thread / OPFS	Manages compact DE442s kernel storage under the precision OPFS directory and downloads the local kernel when required.
precision_engine.js	Main thread CSPICE/WASM	Loads DE442s, checks coverage, performs compact precision state-vector calls, and exposes Geometry Finder eclipse windows when ready.
zenith_storage.js	Main thread / OPFS	Manages the ZENITH DE441 package, required files, split kernel discovery, status checks, object-URL export, and package diagnostics.
zenith_downloader_worker.js	Worker	Performs ranged package downloads, parallel chunking, partial file writes, and hash/size verification during package installation.
zenith_engine.js	Main thread bridge	Creates the worker runtime, routes calculate/stateVector requests, handles timeouts/caching, and falls back to main-thread runtime when needed.
zenith_worker.js	Worker	Loads CSPICE/WASM, mounts kernel files, computes state vectors, and returns results or logs to the main bridge.
reloaded_labs.js	Main thread / lab controller	Runs validation, builds dossiers, performs ET sweeps, solves Mission Lab transfer grids, computes metrics, and renders plots.
three_mode.js	Main thread WebGL	Loads textures, builds the 3D scene, manages camera/focus/orbits, and synchronizes display telemetry.

SECTION 04

Startup & Engine Routing

The browser shell loads in layers so that the analytical path is available quickly while precision services come online only when their assets and runtime hooks are ready.

Order	Loaded component	Purpose
1	three.js, OrbitControls, GSAP	External rendering and animation dependencies used by the 3D telemetry layer and UI transitions.
2	vsop87_data.js, exoplanets.js, vsop_worker.js	Analytical data and survey engine path available before precision packages finish loading.
3	zenith_config.js	Defines package and Horizons proxy endpoints plus compact DE442s kernel URL.
4	precision_storage.js, precision_engine.js	Adds compact precision installation and runtime capability.
5	zenith_storage.js, zenith_engine.js	Adds DE441 package storage, CSPICE/WASM runtime loading, and worker-backed ZENITH calls.
6	tz-lookup.js, ui.js	Adds timezone lookup and primary interface behavior.
7	three_mode.js, reloaded_labs.js	Adds 3D telemetry, validation lab, and Mission Lab after base UI exists.

READINESS RULE The application does not treat loaded page as equivalent to precision-ready. Runtime services expose status, installed files, missing files, and error states so the UI can show the correct operating posture.

SECTION 05

Precision Asset Pipeline

Horizon 87 uses the browser Origin Private File System as a local precision-asset store. The ZENITH package is larger and more complex than the compact DE442s path, so it uses a package manager plus a parallel range-download worker.

- **DE442s compact path:** a single compact kernel stored under a dedicated precision directory for quicker local precision access.
- **ZENITH package:** split DE441 kernel parts, leap-second kernel, cspice.wasm, cspice.js, and metadata managed under a separate OPFS root.
- **Downloader strategy:** metadata fetch, HEAD size check, parallel byte-range chunk requests, temporary .part files, and completion verification.
- **Integrity posture:** file-size checks everywhere; SHA-256 verification where package metadata provides expected hashes.
- **Reuse model:** once installed, files are reused offline for local computation until removed or replaced.

Asset group	Files / location	Engineering role
DE442s compact store	de442s.bsp in horizon87-precision	Compact precision kernel for coverage-bounded precision calls.
ZENITH package root	de441_part-1.bsp, de441_part-2.bsp, naif0012.tls, cspice.wasm, cspice.js, zenith_metadata.json	Full local DE441 + CSPICE runtime package used by the ZENITH path.
Downloader worker	Parallel range-fetch pipeline in zenith_downloader_worker.js	Improves large-package installation while keeping the UI responsive.
Status / diagnostics	Installed-files check, missing-files check, quota reporting, and metadata inspection	Lets the UI report exact package posture instead of a vague not-ready label.

SECTION 06

ZENITH Runtime & Worker Flow

ZENITH is the local DE441 / CSPICE state-vector runtime. Its engine bridge attempts worker-first execution while maintaining a main-thread fallback for environments where the worker path fails but the CSPICE runtime can still load.

Component	Behavior	Failure handling
Runtime loader	Loads cspice.js and cspice.wasm from OPFS object URLs. Supports module-style Emscripten factories and script-tag fallback loading.	Throws if no supported SPK state adapter is found.
Kernel mount	Mounts DE441 and leap-second files through supported runtime hooks or raw module paths.	Fails clearly when the runtime exposes no supported kernel loader.
Worker initialization	Creates a named worker, sends runtime URLs and kernel files, then waits for ready/result/error messages.	If worker initialization fails, the engine terminates it and attempts main-thread runtime fallback.
Request routing	stateVector and calculate requests carry target, observer, correction, ET/JDE, optional ET offset, unique IDs, and payload keys.	Requests include timeouts; stale pending keys are cleared on error or result.
State-vector call	Calls a SPICE-compatible state function using target, ET, J2000, aberration correction, and observer.	Rejects invalid vectors and asynchronous adapters where synchronous rendering requires immediate results.

Worker message contract: Main to worker: init, calculate, stateVector. Worker to main: ready, result, error, log. Each calculation request uses an ID so the main bridge can resolve the matching promise, clear timeouts, and cache the latest result for display-synced rendering.

SECTION 07

Validation Lab Flow

The validation lab lives in reloaded_labs.js. It builds a live comparison between NASA/JPL Horizons VECTORS rows and ZENITH CSPICE state vectors.

1. User selects a fixture or validation date set.
2. The lab requests a local ZENITH state vector under explicit target, observer, frame, time scale, and correction conventions.
3. A Horizons reference row is fetched through the configured proxy/API path.
4. The lab normalizes units and computes Cartesian, range, angular, radial, and tangential residuals.
5. Records are summarized into maxima, per-fixture results, gate checks, and a precision summary.
6. A dossier export is produced with methodology text, package status, ET sweep results, and the full residual record set.

LEGACY FIELD NOTE The `strictMode` flag is a legacy runtime failsafe from an older architecture. It is not the validation criterion for the public report. Validation authority comes from the matched Horizons rows, local ZENITH vectors, pass counts, precision summary, and residual thresholds.

SECTION 08

Mission Lab Flow

The Mission Lab is a preliminary interplanetary transfer screening pipeline. It reads origin/target/date controls, builds departure and arrival grids, obtains body states, solves Lambert transfers, computes energy metrics, filters invalid cells, and renders porkchop plots.

Subsystem	Engineering function
Configuration	Reads origin, target, transfer branch, date windows, time-of-flight bounds, C3 / arrival limits, selected metric, resolution, and debug overlay settings.
State sampling	Samples heliocentric body states from the selected engine path and estimates body velocities when needed for v-infinity calculations.
Lambert solver	Uses a universal-variable Lambert formulation with Stumpff functions, rejecting impossible or unstable transfer cells.
Metrics	Computes total v-infinity, C3, departure v-infinity, arrival v-infinity, approximate total delta-v, and time of flight.
Grid integrity	Tracks valid/invalid cells, branch discontinuities, color integrity, and grid metadata for rendering and export.
Rendering	Paints porkchop plots and best-candidate overlays for fast mission-window screening.

SECTION 09

2D UI & Observer Systems

The main interface layer coordinates date/time inputs, time conversions, engine switching, geocentric/heliocentric rendering, scanners, observer guidance, and user-feedback panels.

Area	Implementation responsibilities
Date and engine controls	Parses date input, maintains the active time basis, and routes requests to the selected analytical / precision / ZENITH engine path.
2D solar-system canvas	Handles geocentric and heliocentric drawing, labels, body positions, and view-state updates.
Observer guide	Combines sky positions with altitude, twilight, timezone, hour angle, elongation, and observer presets to produce human-facing event guidance.
Scanner controls	Coordinates planet parade modes, event filters, scan data exports, moon phases, eclipse logic, and toast/status messaging.
Location / timezone assets	Uses timezone lookup and country geometry data to support observer presets and local event interpretation.

The UI favors transparency over a single universal claim. Engine cards, ZENITH status, validation panels, and Mission Lab outputs show which subsystem is active, which trust band applies, and whether the necessary precision assets are installed.

SECTION 10

3D Renderer Flow

The 3D renderer is an interpretive WebGL layer. It receives active ephemeris positions and projects them into a navigable scene with textured bodies, orbits, camera focus transitions, and HUD updates. It does not independently certify ephemeris accuracy.

Feature	Engineering behavior
Texture system	Loads body texture assets where available and creates procedural/fallback texture surfaces when assets are missing.
Body construction	Builds major bodies with radii, physical metadata, materials, clouds/night lights where relevant, Saturn rings, atmosphere layers, and labels/HUD hooks.
Coordinate conversion	Maps ecliptic longitude/radius and optional inclination/node data into Three.js Cartesian positions using log-scaled distance.
Orbits	Generates orbital geometry for planets and the Moon, rebuilds/syncs orbit lines, and calibrates lines to body positions.
Focus controls	Supports pointer interaction, double-click focus, camera transitions, OrbitControls damping, and flight/follow behavior.
Exposure and timing	Updates auto-exposure, frame timing, and render synchronization while clamping frame gaps where needed.

SECTION 11

Public Repository Boundaries

The public repository is documentation-forward. Its purpose is to let reviewers understand the system, inspect validation artifacts, and see public-facing materials without disclosing every production asset or deployment detail.

Includes	Excludes
Documentation PDFs, screenshots, validation exports, release notes, and selected non-production excerpts or diagrams.	Full production source code, private deployment configuration, private runtime package logic, and large precision kernel package files.
A structured README and docs/ layout that act as the 30-second entry point to the project.	Claims of affiliation with NASA, JPL, NAIF, or any space agency.
validation/ summaries and a short reading guide for exported JSON / CSV files.	Anything implying that visual outputs automatically inherit the strongest validation claim.

DISCLOSURE LANGUAGE *The repository should state clearly that Horizon 87 is not affiliated with NASA, JPL, NAIF, or any space agency; it uses those ecosystems for public reference data and kernel/toolkit comparison.*

SECTION 12

Engineering Constraints & Future Notes

The V1 architecture is intentionally scoped to desktop browser operation. It supports a serious local precision path, but it inherits limitations from WebAssembly runtime loading, OPFS availability, browser quota, large kernel package size, and live-reference network dependency.

Constraint	Impact	V1 posture
Desktop browser target	Large canvases, sidebars, WebGL, and OPFS-backed package installation are not optimized for mobile.	Mobile is not the primary release target.
WebAssembly dependency	CSPICE runtime requires WASM and compatible runtime glue.	Fallback paths exist, but validation-grade local state vectors need CSPICE support.
OPFS dependency	Precision kernels must persist in origin-scoped private storage.	Install/status flows report support, missing files, installed files, and quota estimates.
Large kernel footprint	DE441 split BSP files consume multiple gigabytes.	Package downloader verifies size/hash and stores locally once.
Live reference dependency	Validation requires NASA/JPL Horizons reference rows via proxy/API path.	Local computation can be offline after installation; live validation cannot.
Mission model scope	Lambert grids omit full spacecraft operations and launch-system constraints.	Mission Lab remains preliminary screening.

Future architecture directions include expanded regression testing, additional validation fixtures, a more explicit public API/documentation surface, richer Mission Lab models, and lighter-weight modes for constrained browsers.